

Report 2: Total Energy Control System (TECS)

Part I: Complete Original System — Every Function

Part II: Every Soaring-Mode Modification and Its Physical Impact

PX4 Autosozaring Implementation — Defence Technical Report

Radhouene — Fixed-Wing Autosozaring Project

April 2026

Contents

List of Symbols	4
I Original TECS System	9
1 Physical Background	9
1.1 Specific Energy Definitions	9
1.2 Two-Channel Decoupling	9
2 Class Architecture	9
3 Class 1: TECSAirspeedFilter	10
3.1 initialize()	10
3.2 update()	10
3.3 getState()	11
4 Class 2: TECSAltitudeReferenceModel	12
4.1 initialize()	12
4.2 update() — The Two Trajectory Generators	12
4.3 getAltitudeReference()	13
4.4 getHeightRateSetpointDirect()	13
5 Class 3: TECSControl — All Functions	13
5.1 resetIntegrals()	13
5.2 initialize()	14
5.3 update()	14
6 Internal Stage 0: _calculateTotalEnergyRateLimit()	15
7 Internal Stage 1: _calcAirspeedControlOutput()	15

8	Internal Stage 2: <code>_calcAltitudeControlOutput()</code>	16
9	Internal Stage 3: <code>_calcSpecificEnergyRates()</code>	16
10	Internal Stage 4: <code>_detectUnderspeed()</code>	16
11	Internal Stage 5: <code>_calcPitchControl()</code> and Sub-functions	17
11.1	<code>_calcPitchControl()</code> Orchestrator	17
11.1.1	Sub-function A: <code>_updateSpeedAltitudeWeights()</code>	17
11.1.2	Sub-function B: <code>_calcPitchControlSebRate()</code>	18
11.1.3	Sub-function C: <code>_calcPitchControlUpdate()</code>	18
11.1.4	Sub-function D: <code>_calcPitchControlOutput()</code>	19
11.1.5	Sub-function E: Pitch Rate Limit and Final Clamp	19
12	Internal Stage 6: <code>_calcThrottleControl()</code> and Sub-functions	19
12.1	<code>_calcThrottleControl()</code> Orchestrator	19
12.2	<code>_calcThrottleControlSteRate()</code>	20
12.3	<code>_calcThrottleControlUpdate()</code>	20
12.4	<code>_calcThrottleControlOutput()</code>	21
13	Class 4: TECS Outer Facade — All Functions	22
13.1	<code>initControlParams()</code>	22
13.2	<code>initialize()</code>	22
13.3	<code>calcTrueAirspeedSetpoint()</code>	23
13.4	<code>_setFastDescend()</code>	23
13.5	<code>update()</code> — Full Outer Loop	23
14	Complete Function Inventory	25
II	Soaring-Mode Modifications and Their Physical Impact	25
15	Overview: 8 Problems Without Soaring Mode	26
16	M1: Freeze Altitude Reference Model in Glide	26
16.1	Problem	26
16.2	Fix: Re-initialize Each Cycle	26
16.3	Impact on Altitude Controller	27
16.4	Impact on Glide Exit	27
17	M2: Gravity-Limited Airspeed Acceleration (Stage 1)	27
17.1	Problem	27
17.2	Fix	27
17.3	Impact	27
18	M3: Underspeed Detection Bypass (Stage 4)	28
18.1	Problem	28
18.2	Fix	28
18.3	Impact	28

19 M4: Energy Weighting Override (<code>_updateSpeedAltitudeWeights</code>)	28
19.1 Problem	28
19.2 Fix	28
19.3 Physical Interpretation	28
20 M5: Throttle Output Hard-Zero (<code>_calcThrottleControlOutput</code>)	29
20.1 Fix	29
20.2 Impact	29
21 M6: Throttle Integrator Exponential Decay (<code>_calcThrottleControlUpdate</code>)	29
21.1 Problem	29
21.2 Fix	29
21.3 Numerical Example	29
22 M7: Load-Factor STE Correction Disabled (<code>_calcThrottleControlSteRate</code>)	30
22.1 Problem	30
22.2 Fix	30
22.3 Impact	30
23 M8: Pitch Integrator Partial Reset at Entry and Exit	30
23.1 Entry Problem	30
23.2 Exit Problem	30
23.3 Fix: Symmetric Partial Reset	30
23.4 Why $\times 0.5$ and not Full Reset?	31
23.5 Impact	31
24 M9: Glide Airspeed Setpoint (<code>calcTrueAirspeedSetpoint</code>)	31
24.1 New Glide Path	31
24.1.1 Why the Old Version Was Wrong	31
24.1.2 New Design	32
24.1.3 Example (SIH 300 g Model)	32
24.2 Three Airspeed Sources	32
24.3 Polar Best-Glide Derivation (Mode 2)	32
24.4 Banked Stall Speed Floor	33
25 M10: Fast-Descend Suppression (<code>_setFastDescend</code>)	33
25.1 Problem	33
25.2 Fix	33
25.3 Impact	33
26 New Public API (Soaring Interface)	33
27 Complete Modification Summary	34

List of Symbols

Throughout this report, the following notation is used consistently. A hat ($\hat{\cdot}$) denotes a Kalman-filter estimate. A superscript $(\cdot)^-$ denotes a *prior* (predicted before measurement update). A superscript $(\cdot)^*$ denotes a *setpoint* (commanded value). A dot ($\dot{\cdot}$) denotes the time derivative.

State Variables

Symbol	Meaning	Unit
V	True airspeed (TAS)	m/s
$\dot{V}$	TAS time derivative	m/s ²
V_{EAS}	Equivalent airspeed (EAS); corrected for air density	m/s
$\dot{V}_{\text{EAS}}$	EAS time derivative	m/s ²
$\hat{V}_{\text{EAS}}$	Kalman-filter estimate of EAS	m/s
$\hat{\dot{V}}_{\text{EAS}}$	Kalman-filter estimate of EAS rate	m/s ²
$\hat{\mathbf{x}}$	Filter state vector ($\hat{V}_{\text{EAS}}, \hat{\dot{V}}_{\text{EAS}}$) [⊤]	—
$\hat{\mathbf{x}}^-$	Prior (predicted) state vector before measurement update	—
h	Altitude above mean sea level (AMSL)	m
$\dot{h}$	Altitude rate (positive = climbing)	m/s
θ	Pitch angle	rad
ϕ	Bank (roll) angle	rad

Setpoints and Reference Signals

Symbol	Meaning	Unit
V^*, V_{TAS}^*	TAS setpoint (commanded)	m/s
V_{EAS}^*	EAS setpoint	m/s
$\dot{V}^*$	Demanded airspeed rate	m/s ²
$\dot{V}_{\text{max}}^*, \dot{V}_{\text{min}}^*$	Upper/lower bounds on demanded airspeed rate	m/s ²
h^*	Altitude setpoint from navigator	m
h_{ref}	Jerk-limited altitude reference from trajectory model	m
$\dot{h}_{\text{ref}}$	Altitude rate of the trajectory reference model	m/s
$\dot{h}^*$	Demanded altitude rate	m/s
θ^*	Pitch setpoint	rad
δ_t	Throttle setpoint (0 = idle, 1 = full)	—
$\delta_{t,\text{pred}}$	Bilinear feed-forward throttle prediction	—
$\tilde{\dot{B}}$	SEB rate control error = $\dot{B}_{\text{sp}} - \dot{B}_{\text{est}}$	m ² /s ³
$\tilde{\dot{E}}$	STE rate control error = $\dot{E}_{\text{STE}}^* - \hat{\dot{E}}$	m ² /s ³

Energy Quantities

Symbol	Meaning	Unit
$E_k = V^2/(2g)$	Specific kinetic energy (per unit weight)	m
$E_p = h$	Specific potential energy	m
$E = E_k + E_p$	Total specific energy	m
$\dot{E}_k = V\dot{V}/g$	Specific kinetic energy rate	m ² /s ³
$\dot{E}_p = \dot{h}$	Specific potential energy rate (altitude rate)	m ² /s ³
$\dot{E} = \dot{E}_k + \dot{E}_p$	Total specific energy rate	m ² /s ³
$\dot{E}_{\max}$	Maximum reachable STE rate (max climb $\times g$)	m ² /s ³
$\dot{E}_{\min}$	Minimum reachable STE rate (max sink $\times g$, negative)	m ² /s ³
$\hat{\dot{E}}$	Low-pass filtered STE rate estimate	m ² /s ³
$\dot{B}$	Specific energy balance (SEB) rate = $w_{\text{spe}}\dot{E}_p - w_{\text{ske}}\dot{E}_k$	m ² /s ³
$\dot{B}_{\text{sp}}$	SEB rate setpoint (from weighted setpoint quantities)	m ² /s ³
$\dot{B}_{\text{est}}$	SEB rate estimate (from weighted measured quantities)	m ² /s ³
$\dot{E}_{p/k,\text{sp}}$	Potential/kinetic SER setpoint component	m ² /s ³
$\dot{E}_{p/k,\text{est}}$	Potential/kinetic SER estimated component	m ² /s ³
$\dot{E}_{\text{STE}}^*$	STE rate setpoint (with load-factor correction)	m ² /s ³

Weighting and Control Gains

Symbol	Meaning	Unit
w_{spe}	Altitude (potential energy) weight on pitch loop, $\in [0, 2]$	—
w_{ske}	Airspeed (kinetic energy) weight on pitch loop, $\in [0, 2]$	—
w_p	Speed priority parameter (FW_T_SPDWEIGHT); $w_{\text{ske}} = w_p$, $w_{\text{spe}} = 2 - w_p$	—
$k_V = 1/\tau_V$	Airspeed proportional gain (airspeed_error_gain)	s ⁻¹
$k_h = 1/\tau_h$	Altitude proportional gain (altitude_error_gain)	s ⁻¹
k_{ff}	Altitude feedforward gain (altitude_setpoint_gain_ff)	—
$K_{i,\theta}$	Pitch integrator gain (integrator_gain_pitch)	—
$K_{i,t}$	Throttle integrator gain (integrator_gain_throttle)	—
K_d (pitch)	Pitch damping gain (pitch_damping_gain)	—
K_{ff} (pitch)	SEB rate feedforward gain (seb_rate_ff)	—
K_{damp}	Throttle damping gain (throttle_damping_gain)	—
K_{STE}	STE rate error to throttle scaler = $1/(\dot{E}_{\max} - \dot{E}_{\min})$	s ³ /m ²
$K_{\delta,+}$	Throttle slope above trim = $(\delta_{t,\max} - \delta_{t,\text{trim}})/\dot{E}_{\max}$	—
$K_{\delta,-}$	Throttle slope below trim = $(\delta_{t,\text{trim}} - \delta_{t,\min})/ \dot{E}_{\min} $	—
K_{lf}	Load-factor STE rate correction gain (load_factor_correction)	m ² /s ³

Integrators

Symbol	Meaning	Unit
I_θ	Pitch integrator state (_pitch_integ_state)	rad
I_t	Throttle integrator state (_throttle_integ_state)	—

Kalman Filter Notation (TECSAirspeedFilter)

Symbol	Meaning	Unit
σ_V	Airspeed measurement noise std. deviation	m/s
$\sigma_{\dot{V}}$	Airspeed <i>rate</i> measurement noise std. deviation	m/s ²
σ_q	Airspeed rate process noise std. deviation (model uncertainty)	m/s ²
K_c	Continuous Kalman gain matrix (2×2) from CARE solution	—
$K_d = \Delta t \cdot K_c$	Discrete Kalman gain matrix	—
ν	Innovation (measurement residual) vector	—
D, C, R_-	Intermediate scalar quantities in CARE solution (defined locally)	—

Physical and Aircraft Parameters

Symbol	Meaning	Unit
$g = 9.81$	Gravitational acceleration	m/s ²
m	Aircraft mass	kg
ρ	Local air density at current altitude	kg/m ³
$\rho_0 = 1.225$	ISA sea-level air density	kg/m ³
$k = k_{\text{EAS2TAS}} = \sqrt{\rho_0/\rho}$	EAS-to-TAS conversion factor (≥ 1)	—
$n_z = 1/\cos \phi$	Normal load factor in banked flight	—
V_{trim}	Trim (cruise) EAS; aircraft flies level at idle trim	m/s
$V_{\text{EAS,min}}, V_{\text{EAS,max}}$	Minimum and maximum EAS flight envelope limits	m/s
$V_{\text{TAS,min}}, V_{\text{TAS,max}}$	Minimum and maximum TAS limits	m/s
$\delta_{t,\text{trim}}$	Throttle at trim speed, level flight	—
$\delta_{t,\text{min}}, \delta_{t,\text{max}}$	Throttle lower/upper bounds	—
$\theta_{\text{min}}, \theta_{\text{max}}$	Pitch angle lower/upper bounds	rad
$a_{z,\text{lim}}$	Vertical acceleration limit (FW_T_VERT_ACC)	m/s ²
$\dot{h}_{\text{climb,max}}$	Maximum achievable climb rate (FW_T_CLMB_MAX)	m/s
$\dot{h}_{\text{sink,min}}$	Minimum sink rate at best glide (positive) (FW_T_SINK_MIN)	m/s
$\dot{h}_{\text{sink,max}}$	Maximum allowable sink rate (FW_T_SINK_MAX)	m/s
$\dot{h}_{\text{climb,target}}$	Target (desired) climb rate, $\leq \dot{h}_{\text{climb,max}}$	m/s
$\dot{h}_{\text{sink,target}}$	Target sink rate, $\leq \dot{h}_{\text{sink,max}}$	m/s

Time Constants and Intervals

Symbol	Meaning	Unit
Δt	Control cycle interval (clamped to [1 ms, 1 s])	s
τ_V	Airspeed error time constant (FW_T_TAS_ERROR_TC, default 10 s)	s
τ_h	Altitude error time constant (FW_T_H_ERROR_TC, default 5 s)	s
τ_{STE}	STE rate LP filter time constant (STE_RATE_TC, default 0.1 s)	s
τ	Throttle integrator decay time constant (FW_GLIDE_I_DECAY)	s
T_{ramp}	Fast-descend ramp-up time (FAST_DESCEND_RAMP_UP_TIME = 2 s)	s
DT_MIN	Minimum cycle time guard = 1 ms	s
DT_MAX	Maximum cycle time guard = 1 s (triggers full reset)	s

Underspeed Detection

Symbol	Meaning	Unit
$r_u \in [0, 1]$	Underspeed ratio: 0 = no underspeed, 1 = fully undersped	—
$p_{\text{err}} = 0.15$	Airspeed error percentage for underspeed boundary computation	—
ε_V	Airspeed error bound = $p_{\text{err}} \cdot V_{\text{trim}}$	m/s
V_{fully}	TAS at which aircraft is fully undersped = $\max(V_{\text{TAS,min}} - 2\varepsilon_V, 0)$	m/s
V_{start}	TAS at which underspeed begins = $\max(V_{\text{TAS,min}} - \varepsilon_V, V_{\text{fully}})$	m/s

Fast-Descend Mode

Symbol	Meaning	Unit
$\alpha_{\text{fd}} \in [0, 1]$	Fast-descend blend factor: 0 = normal, 1 = full fast-descend	—
$h_{\text{err,fd}}$	Altitude error threshold activating fast-descend (FW_T_FAST_ALT_ERR)	m

Trajectory Generators (TECSAltitudeReferenceModel)

Symbol	Meaning	Unit
j_{max}	Maximum jerk of trajectory generator (<code>jerk_max</code>)	m/s ³
a_{max}	Maximum vertical acceleration (<code>vert_accel_limit</code>)	m/s ²
δh	Altitude error to setpoint = $h_{\text{nav}}^* - h_{\text{traj,current}}$	m
h_{vel}	Altitude integrated from velocity generator	m
$h_{\text{traj,current}}$	Current position of altitude-control trajectory generator	m

Glide Polar (Part II)

Symbol	Meaning	Unit
a	Induced drag coefficient of glide polar; $w_s = aV^2 + b$	s ² /m ² (or 1/(m·s))
b	Profile drag coefficient of glide polar	m/s
$w_s(V)$	Sink rate at airspeed V ; $w_s = aV^2 + b$ (positive downward)	m/s
$V_{\text{EAS,polar}}^*$	Best-glide EAS from polar minimum = $\sqrt{b/a}$	m/s
$(L/D)_{\text{max}}$	Maximum lift-to-drag ratio = $1/(2\sqrt{ab})$	—
$V_{\text{TAS,stall,banked}}$	Banked stall TAS = $V_{\text{TAS,min}}\sqrt{n_z}$	m/s

Mathematical Operators

Operator	Meaning	Notes
<code>clamp(x, a, b)</code>	Saturate: $\max(a, \min(b, x))$	Also written <code>constrain</code> in PX4
<code>lerp(a, b, t)</code>	Linear interpolation: $(1 - t)a + tb$	$t \in [0, 1]$; also <code>lerp()</code> in PX4
<code>sign(x)</code>	Signum: +1 if $x > 0$, -1 if $x < 0$, 0 if $x = 0$	
ε	Positive float machine epsilon / guard value	Used in division denominators
<code>max(a, b), min(a, b)</code>	Standard maximum / minimum	
<code> x , fabsf(x)</code>	Absolute value	

Part I

Original TECS System

1 Physical Background

TECS (Total Energy Control System), originally proposed by A. A. Lambregts (1983), controls a fixed-wing aircraft's airspeed and altitude simultaneously by separating them into two energy channels.

1.1 Specific Energy Definitions

Let m = aircraft mass, $g = 9.81 \text{ m/s}^2$, V = true airspeed, h = altitude. Specific energies (per unit weight mg):

$$E_k = \frac{V^2}{2g} \quad [\text{m, kinetic}] \quad (1)$$

$$E_p = h \quad [\text{m, potential}] \quad (2)$$

$$E = E_k + E_p \quad [\text{m, total}] \quad (3)$$

Specific energy rates $[\text{m}^2/\text{s}^3]$:

$$\dot{E}_k = \frac{V\dot{V}}{g}, \quad \dot{E}_p = \dot{h}, \quad \dot{E} = \dot{E}_k + \dot{E}_p \quad (4)$$

Specific energy balance rate (SEB):

$$\dot{B} = w_{\text{spe}} \dot{E}_p - w_{\text{ske}} \dot{E}_k \quad (5)$$

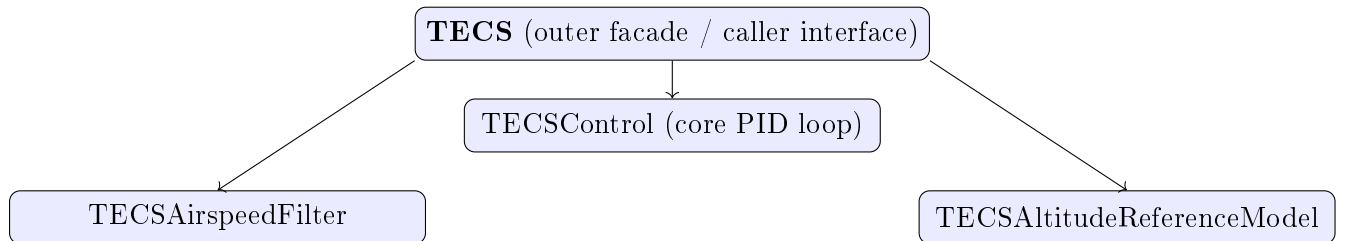
with $w_{\text{spe}} + w_{\text{ske}} = 2$.

1.2 Two-Channel Decoupling

Energy quantity	Controlled by	Physical action
$\dot{E} = \dot{E}_k + \dot{E}_p$	Throttle	Add/remove total energy
$\dot{B} = w_{\text{spe}} \dot{E}_p - w_{\text{ske}} \dot{E}_k$	Pitch	Redistribute energy

2 Class Architecture

Four cooperating C++ classes:



The outer TECS facade owns one instance of each submodule and is the only class called by `FixedwingPositionControl`.

3 Class 1: TECSAirspeedFilter

3.1 initialize()

```
1 void TECSAirspeedFilter::initialize(  
2     const float equivalent_airspeed,  
3     const float equivalent_airspeed_trim,  
4     const bool  airspeed_sensor_available)
```

Called when TECS is first started or after a long update gap ($\Delta t > \text{DT_MAX} = 1 \text{ s}$).

- If sensor available AND EAS is finite: $\hat{V}_{\text{EAS}} \leftarrow \text{equivalent_airspeed}$.
- Otherwise: $\hat{V}_{\text{EAS}} \leftarrow \text{equivalent_airspeed_trim}$ (trim fallback prevents NaN initialisation).
- Always: $\hat{V}_{\text{EAS}} \leftarrow 0$.

Effect: On the first update cycle after reset, the filter starts from a known-good state rather than an uninitialised value.

3.2 update()

Called every TECS cycle with:

- `dt` = elapsed time [s].
- `input.equivalent_airspeed` = raw EAS measurement.
- `input.equivalent_airspeed_rate` = raw EAS rate ($\dot{V}_{\text{EAS}}$ from forward-filtered accelerometer).
- `param` = Kalman noise parameters.
- `airspeed_sensor_available` = validity flag.

Input guard: If `dt` is not a valid timestamp (`TIMESTAMP_VALID(dt)` fails), the function returns immediately with a warning and the state is unchanged.

Sensor fallback:

- If sensor unavailable or EAS not finite: `measurement` $\leftarrow V_{\text{trim}}$, `rate measurement` $\leftarrow 0$.

Prediction step (constant-rate model):

$$\hat{\mathbf{x}}^- = \begin{pmatrix} \hat{V} + \Delta t \hat{\dot{V}} \\ \hat{\dot{V}} \end{pmatrix} \quad (6)$$

Kalman gain (solved from continuous Riccati equation): Define:

$$\sigma_V^{-1} = 1/\sigma_V \quad (\text{speed noise inverse}) \quad (7)$$

$$R_- = \sigma_{\dot{V}}^{-2} \sigma_q \quad (\text{rate-noise-normalised process}) \quad (8)$$

$$D = \sigma_V^{-1} + R_- \quad (9)$$

$$C = \sqrt{\sigma_q (2\sigma_V^{-1} + R_-)} \quad (10)$$

The 2×2 discrete gain is $K_d = \Delta t \cdot K_c$:

$$K_d = \Delta t \cdot \begin{pmatrix} \sigma_V^{-1} C/D & R_-/D \\ \sigma_V^{-2} \sigma_q/D & R_- C/D \end{pmatrix} \quad (11)$$

Innovation vector:

$$\boldsymbol{\nu} = \begin{pmatrix} V_{\text{meas}} - \hat{V}^- \\ \dot{V}_{\text{meas}} - \hat{\dot{V}}^- \end{pmatrix} \quad (12)$$

Update step:

$$\hat{\mathbf{x}} = \hat{\mathbf{x}}^- + K_d \boldsymbol{\nu} \quad (13)$$

Airspeed floor and rate correction: If the updated speed $\hat{V} < \varepsilon$ (negative due to noise):

- Set $\hat{V} \leftarrow 0$.
- Back-calculate the innovation that *would* produce $\hat{V} = 0$:

$$\nu_V^{\text{zero}} = \frac{-\hat{V}^-/\Delta t - K_d(0, 1) \nu_{\dot{V}}}{K_d(0, 0)} \quad (14)$$

- Recompute $\hat{\dot{V}}$ using this adjusted innovation (prevents physically inconsistent negative-speed states with non-zero acceleration).

State update:

$$\text{_airspeed_state.speed} \leftarrow \hat{V} \quad (15)$$

$$\text{_airspeed_state.speed_rate} \leftarrow \hat{\dot{V}} \quad (16)$$

3.3 getState()

Returns the private `\_airspeed\_state` struct (speed + speed_rate) for use by the TECS facade when building the `TECSControl::Input`:

$$V_{\text{TAS}} = k \cdot \hat{V}_{\text{EAS}}, \quad \dot{V}_{\text{TAS}} = k \cdot \hat{\dot{V}}_{\text{EAS}} \quad k = k_{\text{EAS2TAS}} \quad (17)$$

4 Class 2: TECSAltitudeReferenceModel

4.1 initialize()

```
1 void TECSAltitudeReferenceModel::initialize(  
2     const AltitudeReferenceState &state)
```

Called on TECS reset or during glide (see Part II, M1).

- NaN guard: if `state.alt` or `state.alt_rate` is not finite, use 0.
- Calls `reset(jerk=0, vel=alt_rate, pos=alt)` on both trajectory generators: `_alt_control_traj_generator` and `_velocity_control_traj_generator`.

After initialisation, both generators report: $h_{\text{ref}} = h$, $\dot{h}_{\text{ref}} = \dot{h}$ (current state).

4.2 update() — The Two Trajectory Generators

```
1 void TECSAltitudeReferenceModel::update(  
2     const float dt,  
3     const AltitudeReferenceState &setpoint,  
4     float altitude, float height_rate,  
5     const Param &param)
```

Input guard: Returns immediately if `dt` invalid.

Common jerk/accel/velocity limits pushed each cycle:

Generator	Limits set
<code>_velocity_control_traj_generator</code>	MaxJerk, MaxAccelUp/Down, MaxVelUp/Down
<code>_alt_control_traj_generator</code>	MaxJerk, MaxAccel, MaxVel

Note: the velocity generator uses inverted sign convention: $\text{MaxVelUp} = \dot{h}_{\text{sink,max}}$, $\text{MaxVelDown} = \dot{h}_{\text{climb,max}}$ (because the velocity generator uses NED z-axis convention).

Two operating modes, selected by `setpoint.alt_rate`:

Mode A: Height-rate control (`alt_rate` is finite)

1. Velocity generator updated: `_velocity_control_traj_generator.update( $\Delta t, \dot{h}^*$ )`
2. This integrates the jerk-limited velocity profile to produce a jerk-limited altitude trajectory: $h_{\text{vel}} = \int \dot{h}_{\text{jerk-limited}} dt$
3. If h_{vel} is finite: altitude setpoint locked (`control_altitude = true`).
4. Otherwise: altitude-control generator stays at current altitude; `_height_rate_setpoint_direct` is set to the velocity generator's current velocity output (passed directly to TECSControl bypassing the altitude P loop).

Mode B: Altitude control (`alt_rate` is NaN)

1. Velocity generator is reset (does not integrate): `reset(0,  $\dot{h}_{\text{meas}}$ ,  $h_{\text{nav}}^*$ )`.
2. Altitude-control generator receives h^* from navigator.
3. Jerk-limited height rate target:

$$\dot{h}_{\text{target}} = \text{sign}(\delta h) \cdot \text{computeMaxSpeedFromDistance}(j_{\text{max}}, a_{\text{max}}, |\delta h|, 0) \quad (18)$$

where $\delta h = h_{\text{nav}}^* - h_{\text{traj,current}}$. This function computes the maximum speed reachable from the current distance with jerk and acceleration limits (minimum-time bang-bang profile).

4. Height rate clamped: $\dot{h}_{\text{target}} \leftarrow \text{clamp}(\dot{h}_{\text{target}}, -\dot{h}_{\text{sink,target}}, \dot{h}_{\text{climb,target}})$
5. Trajectory generator advanced: `updateDurations + updateTraj`.
6. `_height_rate_setpoint_direct = NAN` (altitude loop will run in `TECSControl` via altitude P controller).

4.3 `getAltitudeReference()`

Returns:

$$(h_{\text{ref}}, \dot{h}_{\text{ref}}) = (\text{_alt_control_traj_generator.getCurrentPosition()}, \text{getCurrentVelocity}()) \quad (19)$$

4.4 `getHeightRateSetpointDirect()`

Returns `_height_rate_setpoint_direct`:

- NaN in altitude-control mode (`TECSControl` uses altitude P loop).
- Current velocity of the velocity generator in height-rate mode (`TECSControl` uses this directly, bypassing the altitude P loop).

5 Class 3: `TECSControl` — All Functions

5.1 `resetIntegrals()`

```
1 void TECSControl::resetIntegrals()
2 {
3     _pitch_integ_state    = 0.0f;
4     _throttle_integ_state = 0.0f;
5 }
```

Zeros both integrators. Called by:

- `TECSControl::initialize()` (cold start / after reset).
- Externally via `TECS::initialize()`.

5.2 initialize()

```
1 void TECSControl::initialize(const Setpoint &setpoint,
2     const Input &input, Param &param, const Flag &flag)
```

Called only on TECS reset (first call or after DT_MAX exceeded). Does **not** run integrators. Its purpose is to compute valid *initial* setpoints so that the *first* normal update cycle does not start from undefined values.

1. `resetIntegrals()`: zero both integrators.
2. Run `_calcAirspeedControlOutput()` to get $\dot{V}^*$.
3. Run `_calcAltitudeControlOutput()` to get $\dot{h}^*$.
4. Compute `SpecificEnergyRates` from these.
5. Run `_detectUnderspeed()` (so r_u is valid).
6. Compute energy weights and SEB rate (no integrator update).
7. Compute initial pitch setpoint θ^* using `_calcPitchControlOutput()` (output only, no integrator).
8. Reset the STE rate LP filter: `_ste_rate_estimate_filter.reset( $\dot{E}_{p,est} + \dot{E}_{k,est}$ )`.
9. Compute initial throttle using `_calcThrottleControlSteRate()` and `_calcThrottleControlOut`.
10. Populate debug output fields.

Key distinction from `update()`: no integrator update is performed. The purpose is *state initialisation*, not control.

5.3 update()

```
1 void TECSControl::update(const float dt,
2     const Setpoint &setpoint, const Input &input,
3     Param &param, const Flag &flag)
```

Input guard: If `dt` invalid, return immediately.

Build `AltitudePitchControl` `control_setpoint`:

1. $\dot{V}^* \leftarrow _calcAirspeedControlOutput()$.
2. Height-rate source selection:
 - If `setpoint.altitude_rate_setpoint_direct` is finite: $\dot{h}^* \leftarrow _altitude_rate_setpoint_direct$ (direct height-rate control: altitude P loop bypassed).
 - Else: $\dot{h}^* \leftarrow _calcAltitudeControlOutput()$ (altitude P loop with FOH feed-forward).

Run the 6 stages:

1. `_calcSpecificEnergyRates()` — build the four SER components.
2. `_detectUnderspeed()` — update r_u .
3. `_calcPitchControl()` — run pitch PID.
4. `_calcThrottleControl()` — run throttle PID.

Debug output fields:

```

1 _debug_output.altitude_rate_control      = control_setpoint.
   altitude_rate_setpoint;
2 _debug_output.true_airspeed_derivative_control = control_setpoint.
   tas_rate_setpoint;
3 _debug_output.pitch_integrator           = _pitch_integ_state;
4 _debug_output.throttle_integrator        = _throttle_integ_state;
```

6 Internal Stage 0: `_calculateTotalEnergyRateLimit()`

```

1 STERateLimit TECSControl::_calculateTotalEnergyRateLimit(
2     const Param &param) const
```

Called at the beginning of Stage 1 (airspeed) and Stage 6 (throttle).

$$\dot{E}_{\max} = \max(\dot{h}_{\text{climb,max}}, \varepsilon) \cdot g \quad [\text{m}^2/\text{s}^3] \quad (20)$$

$$\dot{E}_{\min} = -\max(\dot{h}_{\text{sink,min}}, \varepsilon) \cdot g \quad [\text{m}^2/\text{s}^3] \quad (21)$$

These define the physically reachable STE rate envelope. $\dot{h}_{\text{climb,max}}$ comes from `FW_T_CLMB_MAX` and $\dot{h}_{\text{sink,min}}$ from `FW_T_SINK_MIN`.

7 Internal Stage 1: `_calcAirspeedControlOutput()`

```

1 float TECSControl::_calcAirspeedControlOutput(
2     const Setpoint &setpoint, const Input &input,
3     const Param &param, const Flag &flag) const
```

No sensor path: If `!flag.airspeed_enabled`: return 0.

Compute speed-rate limits from energy limits:

$$\dot{V}_{\max}^* = \frac{(0.5 \alpha_{\text{fd}} + 0.5) \dot{E}_{\max}}{\max(V, \varepsilon)} \quad (22)$$

$$\dot{V}_{\min}^* = \frac{(0.5 \alpha_{\text{fd}} + 0.5) \dot{E}_{\min}}{\max(V, \varepsilon)} \quad (23)$$

$\alpha_{\text{fd}} \in [0, 1]$ is the fast-descend blend factor. At $\alpha_{\text{fd}} = 0$ (normal flight), the factor is 0.5, giving a 50% margin relative to the physical limit.

Proportional speed error:

$$\dot{V}^* = \text{clamp}\left[(V^* - V) k_V, \dot{V}_{\min}^*, \dot{V}_{\max}^*\right] \quad (24)$$

$k_V = 1/\tau_V$ (airspeed_error_gain, default $\tau_V = 10$ s).

8 Internal Stage 2: `_calcAltitudeControlOutput()`

```
1 float TECSCControl::_calcAltitudeControlOutput(  
2     const Setpoint &setpoint, const Input &input,  
3     const Param &param) const
```

Proportional altitude outer-loop with feedforward:

$$\dot{h}^* = k_h (h_{\text{ref}} - h) + k_{\text{ff}} \dot{h}_{\text{ref}} \quad (25)$$

$k_h = 1/\tau_h$ (altitude_error_gain, default $\tau_h = 5$ s), $k_{\text{ff}} = \text{altitude_setpoint_gain_ff}$.
Clamped: $\dot{h}^* \leftarrow \text{clamp}(\dot{h}^*, -\dot{h}_{\text{sink,max}}, \dot{h}_{\text{climb,max}})$.

Note: This function is *only* called when `altitude_rate_setpoint_direct` is NaN.
When finite, the reference model's height rate is used directly (see `TECSCControl::update()`, Stage 2).

9 Internal Stage 3: `_calcSpecificEnergyRates()`

```
1 SpecificEnergyRates TECSCControl::_calcSpecificEnergyRates(  
2     const AltitudePitchControl &control_setpoint,  
3     const Input &input) const
```

Computes all four specific energy rate components:

$$\dot{E}_{p,\text{sp}} = \dot{h}^* \cdot g \quad [\text{m}^2/\text{s}^3] \quad (26)$$

$$\dot{E}_{k,\text{sp}} = V_{\text{TAS}}^* \cdot \dot{V}^* \quad (27)$$

$$\dot{E}_{p,\text{est}} = \dot{h} \cdot g \quad (28)$$

$$\dot{E}_{k,\text{est}} = V_{\text{TAS}} \cdot \dot{V}_{\text{TAS}} \quad (29)$$

Returns `SpecificEnergyRates` struct with `spe_rate{setpoint, estimate}` and `ske_rate{setpoint, estimate}`.

10 Internal Stage 4: `_detectUnderspeed()`

```
1 void TECSCControl::_detectUnderspeed(  
2     const Input &input, const Param &param, const Flag &flag)
```

Early exits: Return with $r_u = 0$ if:

- `!flag.detect_underspeed_enabled`
- `!flag.airspeed_enabled`

Cruise-based boundaries:

$$\varepsilon_V = p_{\text{err}} \cdot V_{\text{trim}} \quad (p_{\text{err}} = 0.15) \quad (30)$$

$$V_{\text{fully}} = \max(V_{\text{min}} - 2\varepsilon_V, 0) \quad (31)$$

$$V_{\text{start}} = \max(V_{\text{min}} - \varepsilon_V, V_{\text{fully}}) \quad (32)$$

Underspeed ratio:

$$r_u = 1 - \text{clamp}\left(\frac{V - V_{\text{fully}}}{\max(V_{\text{start}} - V_{\text{fully}}, \varepsilon)}, 0, 1\right) \quad (33)$$

$r_u = 0$: no underspeed. $r_u = 1$: fully undersped ($V \leq V_{\text{fully}}$).

Effects of $r_u > 0$:

- Throttle output: $\delta_t \leftarrow r_u \delta_{t,\text{max}} + (1 - r_u) \delta_t$.
- Throttle integrator: update is zeroed ($\times(1 - r_u)$ factor).
- Energy weighting: $w_{\text{ske}} \leftarrow 2r_u + (1 - r_u)w_{\text{ske}}$ (ramps toward full speed priority).

11 Internal Stage 5: `_calcPitchControl()` and Sub-functions

11.1 `_calcPitchControl()` Orchestrator

```
1 void TECSControl::_calcPitchControl(
2     float dt, const Input &input,
3     const SpecificEnergyRates &specific_energy_rates,
4     const Param &param, const Flag &flag)
```

Calls the four sub-functions in order:

1. `_updateSpeedAltitudeWeights()` — compute $w_{\text{ske}}, w_{\text{spe}}$.
2. `_calcPitchControlSebRate()` — compute $\dot{B}_{\text{sp}}, \dot{B}_{\text{est}}$.
3. `_calcPitchControlUpdate()` — integrate I_θ .
4. `_calcPitchControlOutput()` — compute θ^* .

Then applies pitch-rate limit and final clamping.

11.1.1 Sub-function A: `_updateSpeedAltitudeWeights()`

```
1 SpecificEnergyWeighting TECSControl::_updateSpeedAltitudeWeights(
2     const Param &param, const Flag &flag)
```

Start from `param.pitch_speed_weight` w_p (FW_T_SPDWEIGHT, range $[0,2]$).

Four conditional overrides (applied in priority order):

1. **Underspeed and airspeed enabled:** $w_p \leftarrow 2r_u + (1 - r_u)w_p$ (ramp toward 2).
2. **No airspeed sensor:** $w_p \leftarrow 0$ (altitude-only pitch control).

3. **Fast descend** ($\alpha_{fd} > 0$): $w_p \leftarrow 1 + \alpha_{fd}$ (speed priority above balanced).

4. **Nominal**: $w_p = \text{parameter}$.

Final weights:

$$w_{ske} = \text{clamp}(w_p, 0, 2), \quad w_{spe} = \text{clamp}(2 - w_p, 0, 2) \quad (34)$$

11.1.2 Sub-function B: `_calcPitchControlSebRate()`

```
1 ControlValues TECSControl::_calcPitchControlSebRate(
2     const SpecificEnergyWeighting &weight,
3     const SpecificEnergyRates &specific_energy_rates) const
```

SEB rate setpoint and estimate:

$$\dot{B}_{sp} = w_{spe} \dot{E}_{p,sp} - w_{ske} \dot{E}_{k,sp} \quad (35)$$

$$\dot{B}_{est} = w_{spe} \dot{E}_{p,est} - w_{ske} \dot{E}_{k,est} \quad (36)$$

The control error is: $\tilde{B} = \dot{B}_{sp} - \dot{B}_{est}$

Positive $\tilde{B}$: more altitude-rate demand relative to speed-rate $\Rightarrow$ pitch up restores balance.

11.1.3 Sub-function C: `_calcPitchControlUpdate()`

```
1 void TECSControl::_calcPitchControlUpdate(
2     float dt, const Input &input,
3     const ControlValues &seb_rate, const Param &param)
```

Normalisation (Lambregts 1983): The sensitivity of SEB rate to pitch angle (small-angle, SPE-dominant):

$$\frac{\partial \dot{B}}{\partial \theta} \approx V_{TAS} \cdot g \quad [\text{m}^2/\text{s}^3 \text{ per rad}] \quad (37)$$

Integrator update:

$$\dot{I}_\theta = \frac{K_{i,\theta} \tilde{B}}{V_{TAS} \cdot g} \quad (38)$$

Anti-windup: do not integrate in the saturating direction:

```
1 if (_pitch_setpoint >= param.pitch_max)
2     pitch_integ_input = min(pitch_integ_input, 0.f);
3 else if (_pitch_setpoint <= param.pitch_min)
4     pitch_integ_input = max(pitch_integ_input, 0.f);
```

Discrete update: $I_\theta[k+1] = I_\theta[k] + \dot{I}_\theta \cdot \Delta t$.

If $K_{i,\theta} \leq \varepsilon$: $I_\theta \leftarrow 0$.

11.1.4 Sub-function D: `_calcPitchControlOutput()`

```

1 float TECSControl::_calcPitchControlOutput(
2     const Input &input, const ControlValues &seb_rate,
3     const Param &param, const Flag &flag) const

```

Airspeed fallback: If sensor unavailable or TAS not finite: $V_{\text{for calc}} \leftarrow V_{\text{trim}} \cdot k_{\text{EAS2TAS}}$.

PD + integrator:

$$\theta_{\text{unc}}^* = \frac{K_d \ddot{B} + K_{\text{ff}} \dot{B}_{\text{sp}}}{V_{\text{TAS}} \cdot g} + I_\theta \quad (39)$$

$K_d = \text{pitch_damping_gain}$, $K_{\text{ff}} = \text{seb_rate_ff}$.

Constrained: $\theta^* = \text{clamp}(\theta_{\text{unc}}^*, \theta_{\text{min}}, \theta_{\text{max}})$.

11.1.5 Sub-function E: Pitch Rate Limit and Final Clamp

Back in `_calcPitchControl()`:

$$\Delta\theta_{\text{max}} = \frac{\Delta t \cdot a_{z,\text{lim}}}{\max(V_{\text{TAS}}, \epsilon)} \quad [\text{rad}] \quad (40)$$

Rate-limited:

$$\theta^*[k] = \text{clamp}(\theta_{\text{unc}}^*, \theta^*[k-1] - \Delta\theta_{\text{max}}, \theta^*[k-1] + \Delta\theta_{\text{max}}) \quad (41)$$

Final parameter clamp: $\theta^* \leftarrow \text{clamp}(\theta^*, \theta_{\text{min}}, \theta_{\text{max}})$.

Debug output: `pitch_integrator`, `energy_balance_rate_estimate/sp`.

12 Internal Stage 6: `_calcThrottleControl()` and Sub-functions

12.1 `_calcThrottleControl()` Orchestrator

```

1 void TECSControl::_calcThrottleControl(
2     float dt, const SpecificEnergyRates &specific_energy_rates,
3     const Param &param, const Flag &flag)

```

1. Update the STE rate LP filter (every cycle).
2. Compute `ste_rate` via `_calcThrottleControlSteRate()`.
3. **Fast-descend early exit:** If $1 - \alpha_{\text{fd}} < \epsilon$ (full fast-descend): $\delta_t \leftarrow \delta_{t,\text{min}}$; skip integrator update.
4. Else: normal path:
 - (a) `_calcThrottleControlUpdate()` — update integrator.
 - (b) Blend with fast-descend:

$$\delta_t = (1 - \alpha_{\text{fd}}) _calcThrottleControlOutput() + \alpha_{\text{fd}} \delta_{t,\text{min}} \quad (42)$$

5. Throttle slew-rate limit: if $|\text{throttle_slebrate}| > \epsilon$:

$$|\Delta\delta_t| \leq \Delta t \cdot (\delta_{t,\text{max}} - \delta_{t,\text{min}}) \cdot s_r \quad (43)$$

STE rate LP filter: Raw estimate:

$$\dot{E}_{\text{raw}} = \dot{E}_{k,\text{est}} + \dot{E}_{p,\text{est}} \quad (44)$$

First-order low-pass filter with time constant τ_{STE} (`ste_rate_time_const`, default 0.1 s):

$$\hat{\dot{E}}[k] = e^{-\Delta t/\tau_{\text{STE}}} \hat{\dot{E}}[k-1] + (1 - e^{-\Delta t/\tau_{\text{STE}}}) \dot{E}_{\text{raw}} \quad (45)$$

12.2 `_calcThrottleControlSteRate()`

```
1 ControlValues TECSCControl::_calcThrottleControlSteRate(
2     const STERateLimit &limit,
3     const SpecificEnergyRates &specific_energy_rates,
4     const Param &param, const Flag &flag) const
```

Setpoint:

$$\dot{E}_{\text{STE}}^* = \dot{E}_{k,\text{sp}} + \dot{E}_{p,\text{sp}} \quad (46)$$

Load-factor correction (powered flight only):

$$\dot{E}_{\text{STE}}^* += K_{\text{lf}} (n_z - 1) \quad \text{if } \neg \text{gliding_mode_enabled} \quad (47)$$

Compensates for induced drag increase in a banked turn ($n_z = 1/\cos\phi$).

Constrain:

$$\dot{E}_{\text{STE}}^* \leftarrow \text{clamp}(\dot{E}_{\text{STE}}^*, \dot{E}_{\text{min}}, \dot{E}_{\text{max}}) \quad (48)$$

Estimate: $\dot{E}_{\text{est}}^* = \hat{\dot{E}}$ (from LP filter).

Returns `ControlValues{setpoint, estimate}`.

12.3 `_calcThrottleControlUpdate()`

```
1 void TECSCControl::_calcThrottleControlUpdate(
2     float dt, const STERateLimit &limit,
3     const ControlValues &ste_rate,
4     const Param &param, const Flag &flag)
```

Gain scaler from STE rate error to throttle:

$$K_{\text{STE}} = \frac{1}{\dot{E}_{\text{max}} - \dot{E}_{\text{min}}} \quad (49)$$

Integrator update (only with airspeed sensor):

$$\Delta I_t = K_{i,t} (\dot{E}_{\text{STE}}^* - \hat{\dot{E}}) \Delta t \cdot K_{\text{STE}} \cdot (1 - r_u) \quad (50)$$

$(1 - r_u)$ factor: underspeed conditions zero the integrator update.

Anti-windup:

```

1 if (_throttle_setpoint >= param.throttle_max)
2     throttle_integ_input = min(0.f, throttle_integ_input);
3 else if (_throttle_setpoint <= param.throttle_min)
4     throttle_integ_input = max(0.f, throttle_integ_input);

```

NaN guard:

$$I_t[k+1] = \text{PX4_ISFINITE}(\Delta I_t) ? I_t[k] + \Delta I_t : 0 \quad (51)$$

If $K_{i,t} \leq \varepsilon$: $I_t \leftarrow 0$.

No-sensor path: If `!airspeed_enabled`: integrator is *not* updated (throttle runs open-loop).

12.4 `_calcThrottleControlOutput()`

```

1 float TECSControl::_calcThrottleControlOutput(
2     const STERateLimit &limit,
3     const ControlValues &ste_rate,
4     const Param &param, const Flag &flag) const

```

Bilinear throttle mapping from STE rate:

$$K_{\delta,+} = \frac{\delta_{t,\max} - \delta_{t,\text{trim}}}{\dot{E}_{\max}} \quad (52)$$

$$K_{\delta,-} = \frac{\delta_{t,\text{trim}} - \delta_{t,\min}}{|\dot{E}_{\min}|} \quad (53)$$

$$\delta_{t,\text{pred}} = \begin{cases} \delta_{t,\text{trim}} + \dot{E}_{\text{STE}}^* K_{\delta,+} & \dot{E}_{\text{STE}}^* \geq 0 \\ \delta_{t,\text{trim}} - \dot{E}_{\text{STE}}^* K_{\delta,-} & \dot{E}_{\text{STE}}^* < 0 \end{cases} \quad (54)$$

Full output:

$$\delta_t = K_{\text{damp}} \tilde{E} K_{\text{STE}} + \delta_{t,\text{pred}} + I_t \quad (55)$$

$$\tilde{E} = \dot{E}_{\text{STE}}^* - \hat{E}, \quad K_{\text{damp}} = \text{throttle_damping_gain}.$$

No-sensor fallback: Add only the positive part of the integrator (prevents throttle reduction when transitioning from sensor-enabled to sensor-disabled without losing altitude gains):

```

1 if (!flag.airspeed_enabled)
2     throttle_setpoint += max(0.0f, _throttle_integ_state);

```

Underspeed emergency blend:

$$\delta_t \leftarrow r_u \delta_{t,\max} + (1 - r_u) \delta_t \quad (56)$$

Final clamp: $\delta_t \leftarrow \text{clamp}(\delta_t, \delta_{t,\min}, \delta_{t,\max})$.

13 Class 4: TECS Outer Facade — All Functions

13.1 initControlParams()

```
1 void TECS::initControlParams(  
2     float target_climbrate, float target_sinkrate,  
3     float eas_to_tas, float pitch_limit_max,  
4     float pitch_limit_min, float throttle_min,  
5     float throttle_setpoint_max, float throttle_trim)
```

Called at the start of every `TECS::update()` cycle before the submodule calls. Pushes per-call runtime limits into the control parameter struct:

Input argument	Stored in
<code>target_climbrate</code>	<code>_reference_param.target_climbrate</code>
<code>target_sinkrate</code>	<code>_reference_param.target_sinkrate</code>
$k \cdot V_{\text{EAS},\min}$	<code>_control_param.tas_min</code>
$k \cdot V_{\text{EAS},\max}$	<code>_control_param.tas_max</code>
<code>pitch_limit_max</code>	<code>_control_param.pitch_max</code>
<code>pitch_limit_min</code>	<code>_control_param.pitch_min</code>
<code>throttle_trim</code>	<code>_control_param.throttle_trim</code>
<code>throttle_setpoint_max</code>	<code>_control_param.throttle_max</code>
<code>throttle_min</code>	<code>_control_param.throttle_min</code>

These values can change each call (e.g. tighter pitch limits near the ground, or different throttle limits in landing configuration).

13.2 initialize()

```
1 void TECS::initialize(const float altitude,  
2     const float altitude_rate,  
3     const float equivalent_airspeed,  
4     float eas_to_tas)
```

Called on first TECS cycle or after `DT_MAX` exceeded. Full cold-start reset:

1. **Reference model:** `_altitude_reference_model.initialize` at $(h, \dot{h})$ — both trajectory generators start from current state.
2. **Airspeed filter:** `_airspeed_filter.initialize` at $(V_{\text{EAS}}, V_{\text{trim}})$.
3. **Build initial setpoint:** $V_{\text{TAS}}^* = V_{\text{EAS}} \cdot k$. $\dot{h}_{\text{direct}} = \dot{h}_{\text{ref}}$ (from model output).
4. **Build initial input:** $\text{TAS} = k \cdot V_{\text{EAS}}$, $\text{TAS rate} = 0$.
5. **TECSControl::initialize()** — compute initial setpoints, reset integrators, initialise LP filter.
6. Clear fast-descend state: $\alpha_{\text{fd}} \leftarrow 0$, $\text{timestamp} \leftarrow 0$.

13.3 calcTrueAirspeedSetpoint()

```
1 float TECS::calcTrueAirspeedSetpoint(  
2     float eas_to_tas, float eas_setpoint)
```

Normal powered flight:

$$V_{\text{TAS}}^* = \text{lerp}(k \cdot V_{\text{EAS}}^*, V_{\text{TAS,max}}, \alpha_{\text{fd}}) \quad (57)$$

At $\alpha_{\text{fd}} = 0$: $V_{\text{TAS}}^* = k \cdot V_{\text{EAS}}^*$. At $\alpha_{\text{fd}} = 1$: $V_{\text{TAS}}^* = V_{\text{TAS,max}}$ (fastest possible for descent).

13.4 __setFastDescend()

```
1 void TECS::_setFastDescend(  
2     const float alt_setpoint, const float alt)
```

Activation condition: Airspeed enabled AND $(h^* + h_{\text{err,fd}}) < h$ (aircraft $h_{\text{err,fd}}$ m above setpoint) AND setpoint is finite:

$$\alpha_{\text{fd}}(t) = \text{clamp}\left(\max\left(\alpha_{\text{fd}}, \frac{t - t_{\text{start}}}{T_{\text{ramp}}}\right), 0, 1\right) \quad (58)$$

$T_{\text{ramp}} = \text{FAST_DESCEND_RAMP_UP_TIME} = 2 \text{ s}$. The max ensures α_{fd} only increases (never decreases during ramp-up).

Deactivation (ramp down): If $\alpha_{\text{fd}} > 0$ and altitude error condition is no longer met (aircraft within $h_{\text{err,fd}}$ of target):

$$\alpha_{\text{fd}} \leftarrow \text{clamp}\left(\frac{h - h^* - 5 \text{ m}}{h_{\text{err,fd}}}, 0, 1\right) \quad (59)$$

The -5 m offset: fast-descend is not fully disabled until the aircraft is within 5 m of the target, providing a smooth transition back to normal pitch control.

Clear: $\alpha_{\text{fd}} = 0$, timestamp cleared.

13.5 update() — Full Outer Loop

```
1 void TECS::update(float pitch, float altitude,  
2     float hgt_setpoint, float EAS_setpoint,  
3     float equivalent_airspeed, float eas_to_tas,  
4     float throttle_min, float throttle_setpoint_max,  
5     float throttle_trim, float pitch_limit_min,  
6     float pitch_limit_max, float target_climbrate,  
7     float target_sinkrate, const float speed_deriv_forward,  
8     float hgt_rate, float hgt_rate_sp)
```

Step 1: Compute Δt .

$$\Delta t = \frac{t_{\text{now}} - t_{\text{prev}}}{10^6} \quad [\text{s}] \quad (60)$$

Step 2: initControlParams(). Push throttle/pitch limits, target climb/sink rates, and EAS limits.

Step 3: DT_MIN guard. If $\Delta t < \text{DT_MIN} = 1 \text{ ms}$: return without update. Prevents division by zero and negligible-interval integrations.

Step 4: DT_MAX / first-call reset. If $\Delta t > \text{DT_MAX} = 1 \text{ s}$ OR $t_{\text{prev}} = 0$ (first call):

$$\text{call } \text{initialize}(h, \dot{h}, V_{\text{EAS}}, k) \quad (61)$$

This handles: firmware restart, long pause in position control, estimator reset.

Step 5: Normal update path (else branch).

1. `_setFastDescend()` — update α_{fd} ; push to `_control_param.fast_descend`.
2. **Airspeed filter update:** build `TECSAirspeedFilter::Input` with EAS measurement and EAS rate $= \dot{V}_{\text{body_fwd}}/k$; call `_airspeed_filter.update()`.
3. **Altitude reference model update:** Three branches:
 - If gliding: call `initialize(h,  $\dot{h}$ )` to freeze at current state (M1, Part II).
 - If full fast-descend ($1 - \alpha_{\text{fd}} < \varepsilon$): call `initialize(h,  $\dot{h}$ )` to reset the model (prevents integrator wind-up against h^* when descending rapidly).
 - Normal: call `update(h*,  $\dot{h}_{\text{sp}}$ )` using navigator setpoint.
4. **Build TECSControl::Setpoint:**
 - `altitude_reference` from `getAltitudeReference()`.
 - `altitude_rate_setpoint_direct` from `getHeightRateSetpointDirect()` (NaN or velocity).
 - `tas_setpoint` from `calcTrueAirspeedSetpoint()`.
5. **Build TECSControl::Input:** $\text{TAS} = k \cdot \hat{V}_{\text{EAS}}$, $\text{TAS rate} = k \cdot \hat{\dot{V}}_{\text{EAS}}$, $\text{altitude} = h$, $\text{altitude rate} = \dot{h}$.
6. `TECSControl::update()`.

Step 6: Populate debug status:

```

1 _debug_status.control = _control.getDebugOutput();
2 _debug_status.true_airspeed_sp =
3   calcTrueAirspeedSetpoint(eas_to_tas, EAS_setpoint);
4 _debug_status.true_airspeed_filtered =
5   eas_to_tas * _airspeed_filter.getState().speed;
6 _debug_status.true_airspeed_derivative =
7   eas_to_tas * _airspeed_filter.getState().speed_rate;
8 _debug_status.altitude_reference =
9   _altitude_reference_model.getAltitudeReference().alt;
10 _debug_status.height_rate_reference =
11   _altitude_reference_model.getAltitudeReference().alt_rate;
12 _debug_status.height_rate_direct =
13   _altitude_reference_model.getHeightRateSetpointDirect();
14 _debug_status.fast_descend = _fast_descend;
```

Step 7: Record timestamp.

14 Complete Function Inventory

Class	Function	Report Section
TECSAirspeedFilter	initialize()	§3.1
	update()	§3.2
	getState()	§3.3
TECSAltitudeReferenceModel	initialize()	§4.1
	update()	§4.2
	getAltitudeReference()	§4.3
	getHeightRateSetpointDirect()	§4.4
TECSControl	resetIntegrals()	§5.1
	initialize()	§5.2
	update()	§5.3
	_calculateTotalEnergyRateLimit()	§6
	_calcAirspeedControlOutput()	§7
	_calcAltitudeControlOutput()	§8
	_calcSpecificEnergyRates()	§9
	_detectUnderspeed()	§10
	_calcPitchControl()	§11
	_updateSpeedAltitudeWeights()	§11.A
	_calcPitchControlSebRate()	§11.B
	_calcPitchControlUpdate()	§11.C
	_calcPitchControlOutput()	§11.D
	_calcThrottleControl()	§12
	_calcThrottleControlSteRate()	§12.1
	_calcThrottleControlUpdate()	§12.2
	_calcThrottleControlOutput()	§12.3
TECS	initControlParams()	§13.1
	initialize()	§13.2
	calcTrueAirspeedSetpoint()	§13.3
	_setFastDescend()	§13.4
	update()	§13.5

Part II

Soaring-Mode Modifications and Their Physical Impact

15 Overview: 8 Problems Without Soaring Mode

#	Problem	Root cause	Observed symptom
1	Ref model diverges	Tracks unattainable h^*	Wind-up; pitch spike on exit
2	Accel limit too high	Powered climb energy used	Pitch transients
3	Underspeed fires	Boundary based on cruise	Bogus throttle boost
4	Throttle non-zero	No glide gate	Engine restarts
5	Throttle integrator winds up	Normal accumulation	Throttle surge on exit
6	LF STE correction active	No gate	Contaminates debug
7	Pitch integrator bias at entry	Cruise altitude bias	Speed overshoot
8	Pitch integrator bias at exit	Speed-hold bias	40–50 deg/s spike

16 M1: Freeze Altitude Reference Model in Glide

16.1 Problem

In `TECS::update()`, the normal path calls `_altitude_reference_model.update(dt, { $h^*$ ,  $\dot{h}_{sp}^*$ }, ...)` where h^* is the navigator altitude (e.g. next waypoint, fixed altitude).

In glide the aircraft cannot produce thrust to climb to h^* , so the trajectory generator winds its internal velocity integrator upward without limit. On power restoration:

- Both trajectory generators hold large wind-up states.
- The altitude P controller produces a maximum-climb-rate demand.
- TECS pitch loop fights this against the speed controller.

16.2 Fix: Re-initialize Each Cycle

```
1 if (_control_flag.gliding_mode_enabled) {  
2     const TECSAltitudeReferenceModel::AltitudeReferenceState frozen{  
3         .alt      = altitude,  
4         .alt_rate = hgt_rate};
```

```

5   _altitude_reference_model.initialize(frozen);
6 }

```

Each glide cycle both trajectory generators are reset to current state: $h_{\text{ref}} = h$, $\dot{h}_{\text{ref}} = \dot{h}$.

16.3 Impact on Altitude Controller

$$\dot{h}^* = k_h (h - h) + k_{\text{ff}} \dot{h} = k_{\text{ff}} \dot{h} \quad (62)$$

Altitude controller output equals current measured sink rate times the feedforward gain — it tracks reality, never integrating against an unattainable target.

16.4 Impact on Glide Exit

Neither trajectory generator has wound up. On power restoration the altitude controller produces a smooth, nominal demand rather than a maximum-climb saturation.

17 M2: Gravity-Limited Airspeed Acceleration (Stage 1)

17.1 Problem

Powered-flight acceleration limit:

$$\dot{V}_{\text{max}}^*|_{\text{powered}} = \frac{0.5 \dot{E}_{\text{max}}}{V} = \frac{0.5 \dot{h}_{\text{climb,max}} g}{V} \quad (63)$$

For $\dot{h}_{\text{climb,max}} = 3 \text{ m/s}$, $V = 10 \text{ m/s}$: $\dot{V}_{\text{max}}^* = 1.47 \text{ m/s}^2$.

In glide, the only energy source for acceleration is gravity. The maximum achievable STE rate from gravity:

$$\dot{E}_{\text{grav,max}} = \dot{h}_{\text{sink,max}} \cdot g = |\dot{E}_{\text{min}}| \quad (64)$$

Using the powered limit allows speed setpoints that demand physically impossible accelerations, driving TECS pitch transients.

17.2 Fix

In `_calcAirspeedControlOutput()`:

```

1  const float available_accel_energy_rate =
2      flag.gliding_mode_enabled
3      ? fabsf(limit.STE_rate_min) // gravity-limited
4      : limit.STE_rate_max;      // powered

```

17.3 Impact

For $\dot{h}_{\text{sink,max}} = 3 \text{ m/s}$, $V = 10 \text{ m/s}$:

$$\dot{V}_{\text{max}}^*|_{\text{glide}} = \frac{0.5 \times 3 \times 9.81}{10} = 1.47 \text{ m/s}^2 \quad (65)$$

The acceleration cap is now physically achievable by pitching down. Pitch transients from the speed controller are eliminated.

18 M3: Underspeed Detection Bypass (Stage 4)

18.1 Problem

Cruise trim at 15 m/s, $p_{\text{err}} = 0.15$, $V_{\text{min}} = 12$ m/s:

$$V_{\text{start}} = 12 - 2.25 = 9.75 \text{ m/s TAS} \quad (66)$$

Glide airspeed ≈ 10.3 m/s TAS: r_u is near zero but non-zero in headwinds, triggering throttle boost and weight interference.

18.2 Fix

```
1 if (flag.gliding_mode_enabled) {  
2     _ratio_undersped = 0.0f;  
3     return;  
4 }
```

18.3 Impact

$r_u \equiv 0$ in glide. No throttle boost, no weighting interference, clean telemetry.

19 M4: Energy Weighting Override (`_updateSpeedAltitudeWeights`)

19.1 Problem

Default $w_p = 1$: $w_{\text{ske}} = w_{\text{spe}} = 1$ (balanced). In glide, pitch must take full responsibility for airspeed — there is no throttle to add energy.

19.2 Fix

```
1 if (flag.gliding_mode_enabled) {  
2     weight.spe_weighting = 0.0f;  
3     weight.ske_weighting = 2.0f;  
4     return weight;  
5 }
```

19.3 Physical Interpretation

With $w_{\text{spe}} = 0$, $w_{\text{ske}} = 2$:

$$\dot{B} = 2 \dot{E}_k - 0 \cdot \dot{E}_p = \frac{2V\dot{V}}{g} \quad (67)$$

Pitch only responds to airspeed error. Altitude is a consequence of the glide polar, not a direct target. This is physically correct for a glider.

20 M5: Throttle Output Hard-Zero (`_calcThrottleControlOutput`)

20.1 Fix

```
1 if (flag.gliding_mode_enabled)
2     return param.throttle_min; // = 0
```

20.2 Impact

TECS throttle is exactly 0 during steady glide. The `FixedwingPositionControl` throttle ramps (Report 3) handle entry/exit transitions.

21 M6: Throttle Integrator Exponential Decay (`_calcThrottleControlUpdate`)

21.1 Problem

In cruise $I_t \approx 0.1\text{--}0.3$. If it persists to glide exit, the first powered cycle sees $\delta_t = \delta_{t,\text{pred}} + 0.25 + \dots$ causing a throttle surge.

21.2 Fix

```
1 if (flag.gliding_mode_enabled) {
2     const float decay = max(param.glide_i_decay, 0.1f);
3     _throttle_integ_state -= dt * _throttle_integ_state / decay;
4     return;
5 }
```

This implements:

$$\boxed{I_t(t) = I_t(0) e^{-t/\tau}} \quad \tau = \text{FW_GLIDE_I_DECAY} \quad (68)$$

Euler discrete approximation: $I_t[k+1] = I_t[k] (1 - \Delta t/\tau)$.

21.3 Numerical Example

$\tau = 10\text{ s}$, $I_t(0) = 0.25$:

$$t = 10\text{ s} : I_t \approx 0.092 \text{ (37\%)} \quad (69)$$

$$t = 30\text{ s} : I_t \approx 0.012 \text{ (< 5\%)} \quad (70)$$

After 30 s of glide, the integrator contributes less than 5% of its original bias.

22 M7: Load-Factor STE Correction Disabled (`_calcThrottleControlSteRate`)

22.1 Problem

In powered banking: $\dot{E}^* += K_{lf}(n_z - 1)$ raises the STE demand to compensate for induced drag. In glide, the throttle is hard-zero regardless, so this correction is meaningless but corrupts the debug STE signal.

22.2 Fix

```
1 if (!flag.gliding_mode_enabled)
2     ste_rate.setpoint +=
3     param.load_factor_correction * (param.load_factor - 1.f);
```

22.3 Impact

STE debug signal correctly represents the glide energy balance. No effect on output (hard-zero throttle). Logging quality improved.

23 M8: Pitch Integrator Partial Reset at Entry and Exit

23.1 Entry Problem

In powered cruise, $I_\theta > 0$ (nose-up altitude trim bias). At glide entry, weights flip to $w_{ske} = 2$, $w_{spe} = 0$. The SEB rate setpoint inverts. The stale I_θ fights the speed control, causing airspeed undershoot of ~ 1 m/s for ~ 5 s.

23.2 Exit Problem

During glide, I_θ accumulates a speed-hold (nose-down) bias. On throttle restoration, the total STE rate rises sharply:

$$\Delta \dot{E} \approx 0.4 \times (30 + 15) = 18 \text{ m}^2/\text{s}^3 \quad (71)$$

The pitch rate limit is:

$$\dot{\theta}_{\max} = \frac{a_{z,\text{lim}}}{\max(V, \varepsilon)} = \frac{7}{10} = 0.7 \text{ rad/s} \approx 40 \text{ deg/s} \quad (72)$$

Observed spike: 40–50 deg/s for ~ 3 s.

23.3 Fix: Symmetric Partial Reset

```

1 if (flag.gliding_mode_enabled && !_prev_gliding_mode) {
2     _pitch_integ_state *= 0.5f;    // glide ENTRY
3 } else if (!flag.gliding_mode_enabled && _prev_gliding_mode) {
4     _pitch_integ_state *= 0.5f;    // glide EXIT
5 }
6 _prev_gliding_mode = flag.gliding_mode_enabled;

```

New private member: `bool _prev_gliding_mode{false}`.

23.4 Why $\times 0.5$ and not Full Reset?

A full reset discards CG offset, elevator trim, and accumulated model error — all still valid across the mode transition. Halving preserves half the trim knowledge while removing the directional bias responsible for the transient.

23.5 Impact

Metric	Before	After
Entry airspeed undershoot	$\sim 1 \text{ m/s}$, 5 s	$\lesssim 0.4 \text{ m/s}$, 2 s
Exit pitch-rate spike	40–50 deg/s	$\lesssim 20 \text{ deg/s}$

24 M9: Glide Airspeed Setpoint (calcTrueAirspeedSetpoint)

24.1 New Glide Path

```

1 if (_control_flag.gliding_mode_enabled) {
2     const float tas_sp =
3         eas_to_tas * _control_param.gliding_airspeed_setpoint;
4     // Floor = FW_AIRSPD_STALL (TAS) or half the banked stall speed,
5     // whichever is larger. Does NOT clamp to tas_min (FW_AIRSPD_MIN).
6     const float tas_stall = eas_to_tas * _control_param.
7         glide_stall_eas;
8     const float glide_floor = math::max(tas_stall,
9         _control_param.tas_min * sqrtf(_control_param.load_factor) * 0.5
10         f);
11     return math::constrain(tas_sp, glide_floor, _control_param.tas_max);
12 }

```

24.1.1 Why the Old Version Was Wrong

The original glide path contained:

```

1 return max(constrain(tas_sp, tas_min, tas_max), tas_banked_min);

```

`tas_min` is derived from `FW_AIRSPD_MIN` (the powered-flight minimum, typically 10 m/s). The `constrain()` call silently floors any glide target below 10 m/s to 10 m/s. Setting `FW_GLIDE_AIRSPD = 6` had no effect — TECS always reported `airspeed_sp = 10.005` ($\text{EAS} \times \text{eas2tas}$). The aircraft responded by pitching to -30° trying to reach the unfulfillable 10 m/s, which is the full pitch limit for the SIH model.

24.1.2 New Design

`tas_min` is no longer applied in glide mode because:

1. Underspeed detection is already bypassed (`_ratio_undersped = 0`), so there is no risk of activating underspeed mitigation at low glide speeds.
2. `FW_AIRSPD_MIN` represents the *powered cruise* stall margin. The appropriate floor for glide is the *stall speed* itself, which is `FW_AIRSPD_STALL`.

The new floor is:

$$V_{\text{floor,TAS}} = \max(k \cdot V_{\text{eas2tas}} \cdot V_{\text{STALL}}, 0.5 \times V_{\text{TAS,min}} \times \sqrt{n_z}) \quad (73)$$

where $k = 1$ (direct stall conversion to TAS).

`FW_AIRSPD_STALL` is already used by `fw_att_control` and `airspeed_selector`. `fw_pos_control` now passes it to TECS via `set_glide_stall_eas()` in `parameters_update()`.

24.1.3 Example (SIH 300 g Model)

Parameter	Old value	Recommended
<code>FW_AIRSPD_MIN</code>	10 m/s	10 m/s (unchanged)
<code>FW_AIRSPD_STALL</code>	7 m/s (default)	4 m/s
<code>FW_GLIDE_AIRSPD</code>	10 m/s	6 m/s
Actual glide SP before fix	10.0 m/s (floored)	—
Actual glide SP after fix	6.0 m/s	6.0 m/s
Pitch in glide	−30 (at limit)	≈ −5 (normal)

24.2 Three Airspeed Sources

Mode	Source set by FW Pos Ctrl	Formula
1	<code>airspeed_cmd</code> or <code>FW_GLIDE_AIRSPD</code>	Fixed EAS
2	Polar best-glide	$V^* = \sqrt{b/a}$ (see below)
3, 4	<code>FW_GLIDE_AIRSPD</code>	Fixed hold

24.3 Polar Best-Glide Derivation (Mode 2)

Parabolic polar: $w_s(V) = aV^2 + b$.

Minimise glide angle w_s/V :

$$\frac{d}{dV} \left(\frac{aV^2 + b}{V} \right) = a - \frac{b}{V^2} = 0 \Rightarrow \boxed{V_{\text{EAS}}^* = \sqrt{b/a}} \quad (74)$$

Maximum glide ratio:

$$(L/D)_{\text{max}} = \frac{V^*}{w_s(V^*)} = \frac{1}{2\sqrt{ab}} \quad (75)$$

24.4 Banked Stall Speed Floor

Lift equation in banked flight:

$$V_{\text{stall,banked}}^2 = \frac{V_{\text{stall,level}}^2}{\cos \phi} \Rightarrow V_{\text{TAS,stall,banked}} = V_{\text{TAS,stall}} \cdot \sqrt{n_z} \quad (76)$$

The floor combines two terms:

$$V_{\text{floor}} = \max \left(\underbrace{k_{\text{eas2tas}} \cdot V_{\text{STALL}}}_{\text{FW_AIRSPD_STALL}}, \underbrace{0.5 \times V_{\text{TAS,min}} \cdot \sqrt{n_z}}_{\text{banked half-margin}} \right) \quad (77)$$

- **FW_AIRSPD_STALL term:** absolute aerodynamic floor — the aircraft must never be commanded below stall speed even in engine-off glide.
- **Banked half-margin:** accounts for load-factor increase in thermal turns. Multiplied by 0.5 so the term is only active when the bank angle is large enough that $\sqrt{n_z} > 2 \cdot V_{\text{STALL}}/V_{\text{TAS,min}}$, i.e. severe bank angles where the geometric margin is tighter than the stall speed itself.
- **Not tas_min:** this is the powered-cruise minimum which is intentionally *not* applied here (see previous subsection).

25 M10: Fast-Descend Suppression (`_setFastDescend`)

25.1 Problem

In glide the aircraft is intentionally descending far below the navigator altitude setpoint. Fast-descend would activate immediately ($h > h^* + h_{\text{err,fd}}$) and command $V \rightarrow V_{\text{max}}$, which is unsafe in unpowered flight.

25.2 Fix

```

1 if (_control_flag.gliding_mode_enabled) {
2     _fast_descend = 0.0f;
3     _enabled_fast_descend_timestamp = 0U;
4     return;
5 }
```

25.3 Impact

$\alpha_{\text{fd}} \equiv 0$ in glide. Airspeed setpoint remains at the glide speed, not the maximum. Pitch loop stays in energy-balance mode.

26 New Public API (Soaring Interface)

```

1 // Enable/disable engine-off (gliding) mode
2 void set_gliding_mode_enabled(bool enabled);
3 bool get_gliding_mode_enabled() const;
4
5 // Set pre-computed glide EAS setpoint [m/s EAS]
6 // Source: FW_GLIDE_AIRSPD (modes 1, 3, 4)
7 //      sqrt(FW_POLAR_B / FW_POLAR_A) (mode 2)
8 //      companion airspeed_cmd (mode 1 companion override)
9 void set_gliding_airspeed_setpoint(float eas);
10
11 // Set throttle integrator decay time constant [s]
12 // Parameter: FW_GLIDE_I_DECAY (default 10 s)
13 void set_glide_i_decay(float tau);
14
15 // Set stall EAS floor for glide airspeed setpoint [m/s]
16 // Parameter: FW_AIRSPD_STALL; called from parameters_update() in
17 //   FwPosCtrl
18 // Stored in glide_stall_eas; used in calcTrueAirspeedSetpoint() as the
19 // hard minimum in place of the powered-cruise tas_min (FW_AIRSPD_MIN).
20 void set_glide_stall_eas(float eas);
21
22 // Set load factor n_z = 1/cos(phi) for banked stall floor
23 // Updated each cycle from vehicle_attitude_poll() in FwPosCtrl
24 void set_load_factor(float nz); // existing setter, reused for stall
25   floor

```

27 Complete Modification Summary

#	Location in code	Original behaviour	Modified behaviour
M1	TECS::update() altitude ref model	Tracks navigator h^*	Frozen at $(h, \dot{h})$ each cycle
M2	_calcAirspeedControlOutput	$\dot{V}_{\max}^*$ from powered $E_{\max}$	$ \dot{E}_{\min} $ (gravity)
M3	_detectUnderspeed	Cruise-trim boundary	$r_u \equiv 0$ (bypassed)
M4	_updateSpeedAltitudeWeights	$w_{\text{ske}} = \text{param}$	$w_{\text{ske}} = 2, w_{\text{spe}} = 0$
M5	_calcThrottleControlOutput	PI controller result	Hard-zero ($\delta_{t,\min}$)
M6	_calcThrottleControlUpdate	Normal PI accumulation	Decays $e^{-t/\tau}$
M7	_calcThrottleControlSteRate	LF correction always	Skipped when gliding
M8	_calcPitchControl	No reset	$\times 0.5$ on entry AND exit
M9	calcTrueAirspeedSetpoint	Mission EAS $\times k$, lerp to max	Glide EAS with n_z stall floor
M10	_setFastDescend	Activates on altitude error	Suppressed entirely